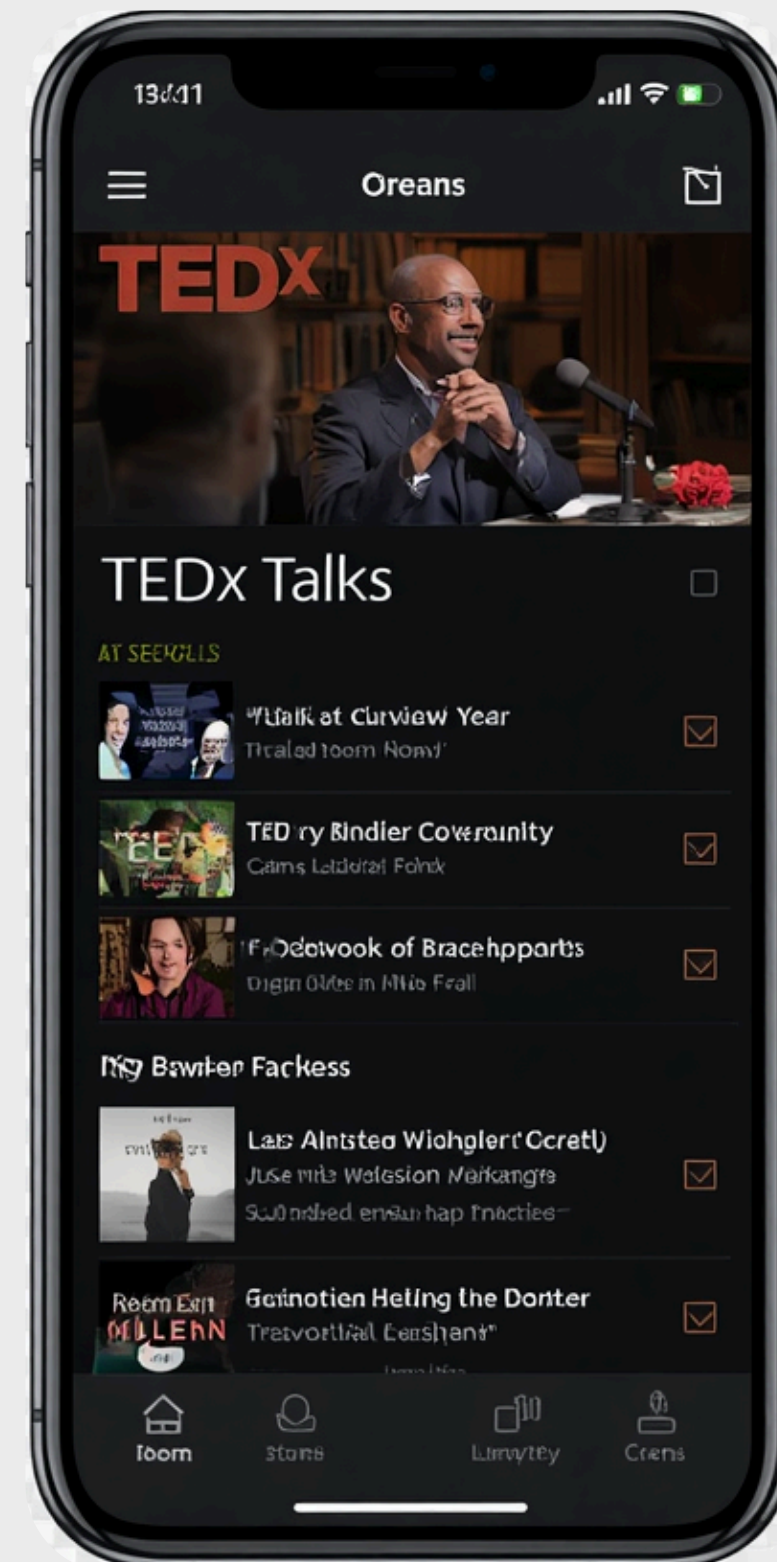


TEDxplore

dai libri alle idee

H3. Integrazione AWS Lambda & API Gateway

Gandolfi Leonardo 1086657
Morgera Claudio 1093069
Zambonelli Julia 1093775



Watch Next by ID

La Lambda riceve l'ID del video corrente e interroga la collezione NoSQL per estrarre i talk correlati.

```
const talk = await talksCollection.findOne(  
  { _id: currentVideoId.toString() },  
  { projection: { related_videos: 1, _id: 0 } }  
);
```

Grazie allo schema denormalizzato, l'estrazione dei video correlati avviene con una singola operazione *findOne*, minimizzando il tempo di esecuzione della Lambda.

Configurazione API Gateway



MAPPING TEMPLATE

Utilizzo di VTL per mappare i parametri dell'URL nell'evento Lambda



Endpoint REST

Implementazione di rotte atomiche per un accesso disaccoppiato alle risorse



CORS Support

Abilitazione degli header
Access_Control-Allow-Origin per il consumo asincrono da Flutter

Efficienza del Nested Schema

Lookup Istantaneo

Grazie al lavoro di denormalizzazione fatto in PySpark, ogni documento contiene già l'array dei video correlati.

Le Lambda Function eseguono una proiezione atomica senza necessità di JOIN a runtime, garantendo un'esperienza fluida a bassa latenza sull'applicazione mobile

Esplorazione per Tag Semantici

Discovery by Tags

La Lambda watch_next_by_tags filtra la collezione tedx_data scansionando l'array dei tag nativi per generare playlist tematiche in tempo reali

Proiezione Mirata

Vengono estratti solo i metadati essenziali, riducendo il payload JSON e i tempi di trasferimento dati verso il client mobile

```
const cursor = talksCollection.find(
  { tags: targetTag },
  { projection: { _id: 1, title: 1, slug: 1, duration: 1 } }
).limit(5);

const tematiciList = [];
await cursor.forEach((doc) => {
  tematiciList.push({
    id: doc._id,
    title: doc.title,
    slug: doc.slug,
    duration: doc.duration
  });
});
```


Algoritmo: Kindle Bridge

Mapping Semantico

Il cuore di TEDxplore: un dizionario che mappa i generi letterari Kindle ai tag TEDx equivalenti.

La Lambda gestisce l'**esclusione dei video già visti** (\$nin) e restituisce una playlist di 5 talk suggeriti, connettendo l'esperienza di lettura con l'ispirazione visiva.

```
// CONNESSIONE E QUERY SU MONGODB ATLAS
const { db } = await connectToDatabase();
const talksCol = db.collection('tedx_data');

const stringWatchedIds = watchedVideoIds.map(id => id.toString());

const query = {
  tags: { $in: uniqueTags },
  _id: { $nin: stringWatchedIds }
};

const projection = { _id: 1, title: 1, slug: 1, url: 1, description: 1, duration: 1 };

const cursor = talksCol.find(query).project(projection).limit(5);
const recommendedTalks = [];

await cursor.forEach((doc) => {
  recommendedTalks.push({
    id: doc._id,
    title: doc.title,
    slug: doc.slug,
    url: doc.url,
    description: doc.description || '',
    duration: doc.duration
  });
});
```

Criticità risolte

01. Connection Pooling

Istanziamento del MongoClient fuori dall'handler per riutilizzare le connessioni TCP ed evitare di saturare MongoDB Atlas.

02. Cold Start

Ottimizzazione del bundle Node.js e delle dipendenze per ridurre i tempi di inizializzazione della Lambda alla prima chiamata.

03. Type Matching

Allineamento tra gli ID trattati come stringhe in API Gateway e i tipi nativi (ObjectId/Int) memorizzati nel database NoSQL.

04. Sicurezza

Gestione degli accessi tramite variabili d'ambiente (MONGO_URI) per isolare le credenziali dal codice sorgente.

Prossimi obiettivi

SECURITY & AUTH

Implementazione di Amazon Cognito per la gestione sicura delle sessioni utente Kindle

EDGE CACHING

Integrazione di CloudFront per servire i suggerimenti con latenza < 50ms

FLUTTER

Realizzazione dell'applicazione vera e propria